

# Abstract

# Contents

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>Abstract</b>                                                                     | <b>1</b> |
| <b>1 Introduction</b>                                                               | <b>5</b> |
| 1.1 Context and Motivation . . . . .                                                | 5        |
| 1.2 Research Objectives . . . . .                                                   | 5        |
| 1.3 Main Contributions of the Thesis . . . . .                                      | 5        |
| 1.4 Structure of the Document . . . . .                                             | 5        |
| <b>2 Theoretical Background</b>                                                     | <b>6</b> |
| 2.1 The Evolution of Learning in Artificial Intelligence . . . . .                  | 6        |
| 2.1.1 Artificial Intelligence Paradigms . . . . .                                   | 6        |
| 2.1.2 Symbolic and Hybrid Approaches in Artificial Intelligence . . . . .           | 8        |
| 2.1.3 Reinforcement Learning: Learning Through Experience . . . . .                 | 9        |
| 2.2 Computer Vision Techniques . . . . .                                            | 10       |
| 2.2.1 Classical Techniques for Video Analysis . . . . .                             | 10       |
| 2.2.2 Visual Perception in Dynamic Environments . . . . .                           | 11       |
| 2.2.3 Deep Learning for Feature Extraction from Video Streams . . . . .             | 12       |
| 2.3 Knowledge Representation and Reasoning . . . . .                                | 15       |
| 2.3.1 Symbolic Representations . . . . .                                            | 15       |
| 2.3.2 Semantic Networks . . . . .                                                   | 16       |
| 2.3.3 Interpretable Models . . . . .                                                | 16       |
| 2.4 Reinforcement Learning Principles . . . . .                                     | 18       |
| 2.4.1 Theoretical foundations . . . . .                                             | 18       |
| 2.4.2 Types of Reinforcement Learning . . . . .                                     | 21       |
| 2.4.3 Deep Q-Networks . . . . .                                                     | 26       |
| 2.5 Atari-like Environments as Testbeds . . . . .                                   | 28       |
| 2.5.1 Characteristics and Challenges of two-dimensional Game Environments . . . . . | 28       |

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>3</b> | <b>State of the Art and Challenges</b>                                              | <b>30</b> |
| 3.1      | Challenges in Symbolic Interpretation . . . . .                                     | 30        |
| 3.1.1    | The Illusion of Understanding . . . . .                                             | 30        |
| 3.1.2    | The Black Box Problem . . . . .                                                     | 33        |
| 3.1.3    | Learning Paradigms and Explainable AI . . . . .                                     | 35        |
| 3.2      | Making Sense of AI: Toward Explainability . . . . .                                 | 37        |
| 3.2.1    | Cognitive Foundations . . . . .                                                     | 37        |
| 3.2.2    | Human World Modelling and the Origins of Structured Cognition . . . . .             | 38        |
| 3.2.3    | Core Knowledge Theory: Cognitive Principles and Psychological Foundations . . . . . | 38        |
| 3.2.4    | Definition and Relevance for World Understanding . . . . .                          | 40        |
| 3.2.5    | Cognitive Models and Symbolic Representation . . . . .                              | 40        |
| 3.3      | Positioning of the Proposed Framework . . . . .                                     | 40        |
| <b>4</b> | <b>Framework Design - Proposed Framework</b>                                        | <b>41</b> |
| 4.1      | Conceptual Foundations . . . . .                                                    | 41        |
| 4.1.1    | Evolution of the Idea . . . . .                                                     | 41        |
| 4.2      | Proposed Implementation . . . . .                                                   | 41        |
| 4.2.1    | Segmentation . . . . .                                                              | 41        |
| 4.2.2    | Symbolic System as Knowledge Construction Module . . . . .                          | 41        |
| 4.2.3    | From Symbolic Rules to a Playable Environment . . . . .                             | 41        |
| 4.2.4    | Generic Environment as a Transitional Representation . . . . .                      | 41        |
| 4.2.5    | Deep Q-Network in a Domain-Agnostic Environment . . . . .                           | 41        |
| <b>5</b> | <b>Implementation</b>                                                               | <b>42</b> |
| 5.1      | End-to-End Pipeline . . . . .                                                       | 42        |
| 5.2      | General Architecture of the Framework . . . . .                                     | 43        |
| 5.2.1    | Design Choices . . . . .                                                            | 43        |
| 5.2.2    | Selected Techniques and Algorithms . . . . .                                        | 44        |
| 5.2.3    | Modular Structure of the System . . . . .                                           | 45        |
| 5.2.4    | Optimizations and Efficiency Considerations . . . . .                               | 48        |
| <b>6</b> | <b>Experimental Evaluation</b>                                                      | <b>49</b> |
| 6.1      | Datasets and Testing Scenarios . . . . .                                            | 49        |
| 6.1.1    | Evaluation Metrics: Accuracy, Interpretability, Scalability . . . . .               | 49        |
| 6.2      | Experimental Results . . . . .                                                      | 49        |
| 6.2.1    | Comparison with Existing Methods . . . . .                                          | 49        |
| 6.2.2    | Critical Discussion of the Results . . . . .                                        | 49        |

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>7 Applications and Future Perspectives - Conclusion</b>                            | <b>50</b> |
| 7.1 Future Directions . . . . .                                                       | 50        |
| 7.1.1 Potential Applications of the Framework . . . . .                               | 50        |
| 7.1.2 Possible Extensions of the Work . . . . .                                       | 50        |
| 7.1.3 Open Challenges and Future Research Directions . . . . .                        | 50        |
| 7.2 Summary of Contributions, Impact of the Research, and Final Reflections . . . . . | 50        |
| <b>References</b>                                                                     | <b>51</b> |
| <b>A Additional Technical Details</b>                                                 | <b>59</b> |
| <b>B Code Snippets or Pseudocode of Relevant Modules</b>                              | <b>60</b> |
| <b>C Dataset Specifications or Experimental Settings</b>                              | <b>61</b> |

# 1 Introduction

## 1.1 Context and Motivation

## 1.2 Research Objectives

## 1.3 Main Contributions of the Thesis

## 1.4 Structure of the Document

## 2 Theoretical Background

The field of artificial intelligence has undergone deep transformations since its inception, evolving through distinct paradigms that reflect different philosophical and technical approaches to machine intelligence. This chapter provides the theoretical foundations necessary to understand the framework proposed in this thesis, exploring the evolution of learning paradigms in AI, some techniques of visual perception and the principles underlying symbolic reasoning and reinforcement learning.

### 2.1 The Evolution of Learning in Artificial Intelligence

#### 2.1.1 Artificial Intelligence Paradigms

The history of artificial intelligence is characterized by the emergence and evolution of three major paradigms, each representing a distinct approach to providing machines the human-like reasoning abilities. The symbolic paradigm, dominant from the 1950s through the 1980s. This paradigm was grounded in the hypothesis that human intelligence could be replicated through the manipulation of symbols, according to logical rules, to perform reasoning tasks. The systems built on this paradigm operated on explicit knowledge representations, using formal logic and rule-based reasoning to solve problems. These systems are grounded in formal logic and manually designed rule sets. Classic examples include expert systems such as MYCIN and Deep Blue, as well as semantic networks and logic programming frameworks [1]. They are capable of performing reasoning tasks and have achieved notable success in constrained domains where knowledge can be explicitly encoded. However, the symbolic paradigm faced significant limitations due to its high reliance on manually encoded knowledge. Indeed, when faced with tasks requiring pattern recognition, learning from experience, the management of uncertainty or incomplete information, it suffered from brittle-

ness [1, 2]. These challenges gave rise to the connectionist paradigm, which gained prominence in the late 1980s and experienced a resurgence in the 2010s with the advent of deep learning [3]. Inspired by the structure and function of biological neural networks, connectionist systems learn representations directly from data through training processes that adjust connection weights between artificial neurons [3, 4]. This paradigm has achieved remarkable success in tasks such as image recognition, natural language processing and game playing, often surpassing human-level performance in specific domains [5]. Their capacity for robust and interpretable reasoning remains limited. Deep learning still struggles with structured reasoning, causal inference and factual consistency. Despite the impressive capabilities of neural networks, their operation and internal representation often lacks transparency, giving rise to what is commonly referred to as the "black box problem" [2], which limits interpretability, accountability and trust. Indeed, many of these approaches compromised interpretability and relied on strong assumptions. Moreover, they lacked the rich semantic expressiveness characteristic of symbolic systems. These limitations provide strong motivation for integrating neural and symbolic paradigms. In short, symbolic systems excel at reasoning but lack at perception, while neural networks excel at perception but struggle with reasoning [1]. This limitation has motivated the emergence of a hybrid paradigm, which seeks to combine the learning capabilities of neural networks with the interpretability and reasoning power of symbolic systems. Neuro-symbolic AI represents this third wave, aiming to bridge the gap between data-driven learning and knowledge-based reasoning as well as between logic-based inference and gradient-based learning. By integrating these paradigms, it aims to potentially offer systems that are both powerful and explainable [6]. Neuro-symbolic methods have demonstrated significant promise across a wide range of application areas, especially in scenarios that demand the combination of data-driven learning and structured symbolic reasoning [6]. This emerging paradigm reflects a synthesis of complementary approaches, bringing together the statistical strengths of deep learning, the formal foundations of symbolic logic and the adaptability of large-scale pretrained models.

Nonetheless, several important challenges remain. These include:

- Ensuring compatibility between the discrete symbolic structures used in symbolic approaches and the continuous vector representations typical of neural networks.
- Enabling the model to dynamically learn, adapt, modify, and refine logical rules.

- Achieving an optimal balance between system complexity and computational efficiency.
- Addressing the absence of unified architectural frameworks by developing a general architecture that systematically and reusably integrates neural and symbolic components.

Although several challenges persist, neuro-symbolic AI is widely regarded as a promising direction for developing systems capable of transparent, reliable and generalizable reasoning.

### **2.1.2 Symbolic and Hybrid Approaches in Artificial Intelligence**

Symbolic AI operates on the principle that intelligence can be reduced to the manipulation of symbols representing concepts, objects and relationships. The knowledge in symbolic systems is typically encoded using various representation schemes, including production rules, semantic networks, frames and logical predicates. These representations enable explicit reasoning processes such as deduction, induction and abduction, allowing systems to draw inferences, explain their decisions and incorporate domain knowledge provided by human experts. The advantages of symbolic approaches include interpretability, the ability to incorporate prior knowledge and support for logical reasoning and explanation. However, they also present significant challenges: knowledge acquisition bottleneck, brittleness when facing situations not covered by explicit rule and difficulty in learning from raw sensory data. The manual encoding of knowledge is intensive and may not capture the full complexity of real-world domains [1]. Hybrid approaches seek to overcome these limitations by integrating symbolic and connectionist methods. Various integration strategies have been proposed, including systems where neural networks generate symbolic representations that are then processed by reasoning engines, architectures that embed symbolic knowledge into neural network structures and frameworks that use symbolic reasoning to guide neural learning processes [6]. Recent work in neuro-symbolic AI has demonstrated promising results in domains requiring both pattern recognition and logical reasoning, such as visual question answering, knowledge graph completion, and program synthesis. Following the advent of neuro-symbolic artificial intelligence, a paradigm shift has been induced in the way machines learn. This approach combines deductive, rule-based learning, typical of symbolic artificial intelligence, with the inductive and pattern-recognition capabilities characteristic of neural networks. The result is a



hybrid approach that leverages the strengths of both domains, leading to a more comprehensive learning methodology. Neuro-symbolic concepts demonstrate strong compositional generalization: new concepts can be created by structurally combining previously learned or existing ones [7]. This compositionality naturally allows for a decomposition of the learning problem, in order to learn and recognize basic concepts. During the inference phase, these concepts can be recombined to achieve the objectives of more complex tasks. Neuro-symbolic AI is one of the most exciting directions in AI research today. It aims to create intelligent systems capable of learning and reasoning like humans.

### **2.1.3 Reinforcement Learning: Learning Through Experience**

Reinforcement learning (RL) represents a distinct learning paradigm in which an agent learns to make decisions by interacting with an environment. Unlike supervised learning, which requires labeled training data, or unsupervised learning, which seeks to discover patterns in unlabeled data, reinforcement learning is based on the principle of learning from the consequences of actions through trial and error [8]. The agent takes actions, observes their consequences, and receives feedback in the form of rewards. At the beginning of learning, an agent may have limited or no prior knowledge of the environment. In model-free approaches, no explicit model is available, and learning relies entirely on interaction. In contrast, model-based methods assume a known model or aim to learn it explicitly. This flexibility makes reinforcement learning suitable for complex environments, where providing a complete dataset or manually specifying optimal behavior is impractical. The core structure of Reinforcement Learning (RL) is modelled through a Markov Decision Process (MDP), which provides a mathematical framework for sequential decision-making [8]. An MDP is defined by a finite or continuous set of states representing the possible situations an agent can encounter, a set of actions available to the agent and a transition function that specifies the probability of moving from one state to another after taking a particular action. Additionally, a reward function assigns numerical feedback to each state-action pair, quantifying the immediate benefit of a given choice. By observing the current state, selecting an action, and receiving a reward and a new state, the agent incrementally improves its decision-making strategy [9]. Over time, the agent aims to learn an optimal policy, which is a strategy mapping states to actions. The goal of this policy is to maximize the expected cumulative reward across future interactions, balancing short-term

gains with long-term benefits through exploration and exploitation. Several agent design approaches exist, including methods based on value estimation and approaches that learn action values directly, such as Q-learning. More advanced techniques combine different strategies to improve learning efficiency. Due to its ability to learn from experience, reinforcement learning has achieved notable success in domains such as game playing, robotics, and resource management [9].

## 2.2 Computer Vision Techniques

Visual perception is fundamental to many AI applications, enabling systems to extract meaningful information from images and video streams. One of the most relevant goals of computer vision is to mimic human visual perception. Robustness is commonly assessed by comparing algorithmic performance to human-level perception under similar conditions. In this context, robustness refers to the ability to extract task-relevant visual information, even when this information is sparse, corrupted, or significantly different from a previously observed representation, while maintaining tolerance to outliers. This section explores both classical and modern approaches to video analysis and feature extraction.

### 2.2.1 Classical Techniques for Video Analysis

Before the advent of deep learning, computer vision systems primarily relied on deterministic techniques and manually engineered features. Traditional algorithms such as edge detection methods, threshold-based segmentation, and morphological processing were widely employed to extract structural information from images. Later, handcrafted feature descriptors including SIFT [10] (Scale-Invariant Feature Transform), SURF [11] (Speeded-Up Robust Features), and HOG [12] (Histograms of Oriented Gradient) were introduced to capture local patterns useful for object recognition and matching tasks. Although these methods achieved notable success, they suffered from several inherent limitations, particularly in their sensitivity to noise, changes in illumination, and poor generalization across diverse environments [10]. As a result, significant domain expertise and extensive manual effort were required to design robust feature representations, which constrained their scalability and adaptability to complex real-world scenarios.

Originally, computer vision relied heavily on handcrafted features and algorithmic techniques designed based on understanding of visual perception and signal processing. These principles underpin a classical computer vision

pipeline, typically composed of preprocessing, feature extraction, and pattern recognition stages.

Preprocessing techniques include operations such as noise reduction, contrast enhancement, and colour space transformations that prepare raw visual data for subsequent analysis. Edge detection algorithms, such as the Canny edge detector and Sobel operator, identify boundaries between regions with different intensity characteristics, providing fundamental structural information about objects in the scene.

Feature extraction methods in classical computer vision include corner detectors like Harris corners [13] and FAST [14] (Features from Accelerated Segment Test), which identify points of interest that can be reliably detected across different views. Descriptors such as SIFT and SURF provide representations of local image patches that are robust to changes in scale, rotation, and illumination. These features enable tasks such as object recognition, tracking, and image matching. Although these classical techniques have been largely superseded by deep learning methods in many applications, they remain valuable in contexts where computational resources are limited, interpretability is crucial, or training data is scarce. Moreover, the principles underlying these techniques continue to inform modern approaches and provide insights into visual perception.

## 2.2.2 Visual Perception in Dynamic Environments

The analysis of dynamic environments is particularly relevant for applications such as autonomous navigation, human-robot interaction, video surveillance, and sports analytics. In the context of video games, which serve as testbeds for this thesis, dynamic environments present controlled yet complex scenarios where agents must perceive, reason about, and respond to changing situations in real-time. Understanding dynamic environments requires not only processing individual images but also tracking changes over time and recognizing temporal patterns. Visual perception in such contexts involves several key challenges: detecting and segmenting objects, tracking their motion across frames, recognizing actions and behaviors, and predicting future states.

Object detection and segmentation are fundamental capabilities for understanding scenes. Object detection identifies the presence and location of objects within images, typically using bounding boxes to localize them. Semantic segmentation assigns a class label to each pixel, providing a more detailed understanding of scene structure. Instance segmentation combines both tasks, identifying individual object instances and their precise boundaries.

Temporal reasoning in video analysis requires integrating information across multiple frames. Recurrent neural networks and their variants, such as Long Short-Term Memory (LSTM) networks [15], have been widely used to capture temporal dependencies. Three-dimensional convolutional networks extend spatial convolutions into the temporal dimension, enabling the extraction of spatiotemporal features directly from video data. In this work, the temporal complexity of video is addressed in a modular manner, by first extracting spatial features from individual frames and delegating temporal modeling to subsequent stages.

### **2.2.3 Deep Learning for Feature Extraction from Video Streams**

The advent of Deep Learning has revolutionized the field of Computer Vision, enabling models to automatically learn hierarchical representations of images and enabling end-to-end learning of feature representations directly from raw pixel data through CNNs (Convolutional Neural Networks).

#### **Convolutional Neural Networks for Visual Representation Learning**

Convolutional Neural Networks (CNNs or ConvNets) form the foundation of modern visual recognition systems and represent a major paradigm shift in computer vision. It enabled models to automatically learn hierarchical representations, directly from raw data, where lower layers capture low-level features such as edges and textures, while higher layers represent more abstract concepts and object categories. Convolutional Neural Networks are designed to process data represented as multidimensional arrays. These structures include one-dimensional signals such as time series and language sequences, two-dimensional data such as images, and three-dimensional data such as videos. The effectiveness of CNNs is rooted in four fundamental architectural principles that exploit the compositional nature of visual data: local connectivity, shared weights, pooling operations, and hierarchical multi-layer organization. Unlike fully connected networks, each convolutional neuron is connected only to a local region of the input, significantly reducing the number of parameters and improving generalization. Weight sharing allows the same convolutional filter to detect a specific visual pattern regardless of its spatial location within the image.

A typical CNN architecture is organized as a sequence of stages, each composed of convolutional layers, nonlinear activation functions, such as the Rectified Linear Unit (ReLU), and pooling layers. In convolutional layers,

neurons are grouped into feature maps, where each map applies a specific filter across the entire input, producing a representation that highlights the presence of particular local patterns. Pooling layers are responsible for locally aggregating information, reducing the spatial dimensionality of feature maps and introducing a degree of invariance to small translations and distortions in the input. The most common operation is max pooling, which selects the maximum value within a local neighborhood. The alternation of convolution, nonlinearity, and pooling enables the network to progressively construct increasingly abstract and robust representations. In the visual domain, low-level features such as edges and orientations are combined into more complex patterns, which in turn form object parts and ultimately complete objects. This hierarchical aggregation allows the learned representations to remain stable under local variations in position, shape, or appearance. In other words, CNNs leverage convolutional operations to capture local spatial patterns, which are gradually combined across deeper layers to form increasingly abstract representations. This hierarchical learning process allows the network to identify complex visual structures without relying on manually designed features. The effectiveness of CNNs was clearly demonstrated in the ImageNet challenge [16], where deep architectures achieved unprecedented performance, significantly outperforming traditional computer vision approaches. This progress has made it possible to tackle complex tasks such as image segmentation, with high accuracy.

## **Evolution of Deep Architectures for Visual Representation Learning**

Architectures for image understanding have evolved considerably since the introduction of AlexNet in 2012 [16]. This model demonstrated the effectiveness of deep convolutional neural networks trained on large-scale datasets such as ImageNet, significantly outperforming traditional computer vision methods and marking a turning point for deep learning in visual recognition tasks. VGGNet [17] further emphasized the importance of network depth, showing that stacking small convolutional filters could improve representational capacity while maintaining manageable computational complexity. Its simple and uniform architecture also made it a widely adopted backbone for transfer learning and feature extraction. ResNet [18] addressed one of the key challenges in training deep networks, namely the vanishing gradient problem. It introduced residual skip connections, which allow gradients to flow more effectively through the network. Furthermore, more recent architectures such as EfficientNet [19] optimize both accuracy and computational efficiency through neural architecture search.

Video understanding requires architectures that can process temporal information. Feature extraction from video streams for downstream tasks often involves using pre-trained networks as feature extractors, transferring knowledge learned on large-scale datasets to specific applications. The features extracted from intermediate or final layers of these networks provide rich representations that can be used for tasks such as action recognition, video classification, and temporal localization of events. The integration of deep learning with computer vision has enabled unprecedented performance in visual recognition tasks, but often at the cost of interpretability. The features learned by deep networks, while effective, do not always correspond to human-understandable concepts, motivating research into explainable AI and the integration of symbolic reasoning with learned representations.

### **From Feature Extraction to Dense Prediction: Semantic Segmentation Architectures**

Early convolutional architectures were primarily developed for image-level classification tasks [19]; however, many real-world applications require a more fine-grained understanding of visual scenes. In particular, tasks such as scene understanding and environment modelling demand dense, pixel-level predictions, where each pixel is assigned a semantic label rather than a single global class. This shift in objectives led to the development of semantic segmentation architectures, which extend the principles of convolutional feature learning to dense prediction problems. A key milestone in this evolution is represented by Fully Convolutional Networks (FCNs) [20], which replace fully connected layers with convolutional ones, enabling networks to process images of arbitrary size and produce spatially structured outputs. Building upon this idea, the U-Net architecture [21] introduced an encoder-decoder structure specifically designed to preserve spatial precision while leveraging deep hierarchical features. The encoder progressively extracts abstract visual representations through convolution and pooling operations, while the decoder reconstructs a dense segmentation map by restoring spatial resolution. A defining characteristic of U-Net is the use of skip connections between corresponding encoder and decoder layers. These connections allow low-level spatial information to be directly combined with high-level semantic representations, mitigating the loss of fine-grained details caused by repeated downsampling. As a result, U-Net achieves accurate boundary localization while maintaining strong semantic consistency [21]. Due to its effectiveness, architectural simplicity, and ability to perform well even with limited training data, U-Net and its variants have been widely adopted across domains, including medical imaging, robotics, and video game analysis. In particu-

lar, U-Net-based models are well suited for environments characterized by structured layouts and clearly defined visual entities, making them a natural choice for perceptual preprocessing in dynamic 2D environments and an effective perceptual backbone for downstream reasoning and decision-making systems.

## **2.3 Knowledge Representation and Reasoning**

The ability to represent knowledge in a structured, manipulable form and to perform reasoning over such representations is central to artificial intelligence. The knowledge representation and reasoning concerned with how knowledge about the world can be formally encoded and manipulated in order to support intelligent behavior. Its primary goal is to design representational formalisms that enable artificial systems to draw logical conclusions, explain their decisions, and reason consistently about complex domains. As emphasized in classical AI literature, effective knowledge representation must balance expressive power, computational efficiency, and interpretability [22]. This section examines approaches to symbolic knowledge representation and their role in building interpretable AI systems.

### **2.3.1 Symbolic Representations**

Symbolic knowledge representation is based on discrete symbols that explicitly encode concepts, entities, and their relationships. This approach stems from a cognitive perspective according to which human reasoning relies on symbolic manipulation, a view that profoundly shaped early artificial intelligence research [?]. Due to both the representations and the inference mechanisms operate on explicit symbols, symbolic systems closely mirror human modes of reasoning and are therefore inherently interpretable. These representations are commonly employed to structure data and metadata, where the semantic meaning of each symbol must be carefully defined and considered during information analysis. Furthermore, since human communication relies predominantly on the use of symbols, symbolic representations also play a fundamental role in the study and analysis of natural language. Symbolic representation involves encoding knowledge using discrete symbols and the relationships between them. Unlike distributed representations in neural networks, where meaning emerges from patterns of activation across many units or is distributed across numerical parameters, symbolic representations

explicitly denote concepts and their properties, encoded in well-defined structures. Each symbol has a clear semantic interpretation, which makes symbolic representations particularly suitable for reasoning, explanation, and the integration of expert knowledge [?].

### 2.3.2 Semantic Networks

Semantic networks are graph-based models designed to organize and represent knowledge through interconnected nodes and labeled edges. In this context, nodes correspond to concepts, objects, or entities, while edges denote semantic relations between them, such as *is-a* (classification), *part-of* (composition), or associative links. This structure allows knowledge to be represented in a way that mirrors conceptual organization, making relationships explicit and visually interpretable. One of the central advantages of semantic networks is their ability to represent taxonomic hierarchies. Through hierarchical relations (e.g.,  $\text{canary} \rightarrow \text{bird} \rightarrow \text{animal}$ ), properties associated with more general categories can be inherited by more specific ones. For instance, if the concept *bird* is associated with the property *has wings*, that property can automatically be inferred for *canary* without being redundantly specified. This mechanism, known as inheritance reasoning, supports efficient knowledge organization and reduces duplication. From a cognitive perspective, semantic networks were strongly influenced by early models of human memory and conceptual organization. A foundational contribution was provided by M. Ross Quillian (1968), who proposed semantic memory models structured as hierarchical networks in which concepts are connected by labeled relations. His work demonstrated how hierarchical organization could explain verification time in human reasoning tasks, suggesting that cognitive processes may operate through structured relational networks. Later, John F. Sowa expanded the formal foundations of semantic networks by integrating them with logic in his theory of Conceptual Graphs, providing a more rigorous semantic interpretation and linking graph structures to formal logic systems. Their graphical notation makes them particularly intuitive, facilitating both human understanding and computational reasoning. Although later symbolic formalisms, such as description logics and ontology languages, provided more formal semantics, semantic networks remain a foundational paradigm in knowledge representation.

### 2.3.3 Interpretable Models

In knowledge representation, a variety of symbolic models have been developed to describe objects, situations, and events in a structured and inter-



pretable way. Among these, frames and scripts extend semantic networks by introducing richer internal structures capable of capturing stereotypical situations and recurring sequences of actions. A frame represents a stereotypical object or situation through a set of attributes, known as slots, which may be filled with specific values, default assumptions, or references to other frames. This structured representation allows frames to encode both declarative knowledge about entities and procedural knowledge about the actions typically associated with a given context. Scripts, by contrast, are specifically designed to model common sequences of events. They describe how actions usually unfold in familiar situations and are particularly effective for narrative understanding and for predicting behavior in routine scenarios. Both models were originally proposed to account for aspects of human comprehension and memory [23]. A more formal and explicit approach to knowledge representation is provided by ontologies, which define a shared conceptual framework for a given domain. Ontologies specify classes, relations, constraints, and axioms using logic-based languages, often grounded in Description Logic. By enforcing precise semantics, they enable automated reasoning tasks such as consistency checking, classification, and logical inference. Their standardized nature also supports knowledge sharing and reuse across different systems, making ontologies a central component of areas such as the Semantic Web, intelligent information retrieval, and large-scale knowledge integration [24, 25]. Another major category of symbolic approaches is logic-based representations, which encode knowledge as sets of formal logical statements. First-order logic provides a powerful framework for expressing complex relationships among entities through predicates, variables, and quantifiers, enabling rigorous deductive reasoning. Alongside this, production rules, typically expressed as if-then statements, capture procedural knowledge and support both forward chaining, in which conclusions are derived from known facts, and backward chaining, in which goals are traced back to the conditions required to satisfy them. These mechanisms have played a foundational role in expert systems and rule-based AI architectures. The strength of logical representations lies in their ability to guarantee consistency, derive logical consequences, and provide clear explanations for inferred conclusions, if inference rules are correctly applied. More broadly, the interpretability of symbolic representations constitutes a key advantage: the meaning of symbols, rules, and relations can be directly understood by humans, facilitating debugging, validation, and trust in AI systems. However, constructing such representations manually or extracting them from raw, unstructured data remains a significant challenge. Despite these limitations, symbolic knowledge representation continues to be a crucial component of artificial intelligence, particularly in applications that require explainabil-

ity, structured reasoning, and formal guarantees. Due to purely symbolic systems are not sufficient on their own, their integration with data-driven approaches offers promising directions for the development of interpretable and robust AI systems. Ongoing research in neuro-symbolic methods aims to automate the extraction of symbolic knowledge from data while preserving the powerful reasoning capabilities that symbolic representations provide.

## 2.4 Reinforcement Learning Principles

This section delves deeper into the principles and methods of reinforcement learning, with particular focus on Deep Q-Networks, which play a central role in the framework proposed in this thesis.

### 2.4.1 Theoretical foundations

The mathematical foundation of reinforcement learning rests on the framework of Markov Decision Processes (MDPs). An MDP formalizes sequential decision-making problems in which an agent interacts with an uncertain environment and seeks to maximize a cumulative reward over time. An MDP is defined by the tuple  $(S, A, P, R, \gamma)$ , where:

- $S$  is the state space, i.e., the set of all possible configurations of the environment.
- $A$  is the action space, i.e., the set of all actions available for the agent in each state.
- $P(s' \mid s, a)$  is the transition probability function, i.e., the probability that, after performing action  $a$  in state  $s$ , the system transitions to state  $s'$ .
- $R(s, a)$  is the reward function, which provides a numerical value quantifying the desirability of transitioning from state  $s$  to state  $s'$  after performing action  $a$ , assigning a scalar value to each state-action pair.
- $\gamma \in [0, 1)$  is the discount factor that determines the relative importance of immediate versus future rewards. Smaller values of  $\gamma$  encourage myopic behavior focused on immediate gains, whereas values closer to 1 promote long-term planning.

A fundamental assumption in MDPs is the Markov property, which states that the future evolution of the system depends solely on the current state

and action, independent of the full history of past states. Mathematically:

$$P(s' \mid s, a, s_{t-1}, a_{t-1}, \dots) = P(s' \mid s, a)$$

This memoryless characteristic significantly simplifies the analysis and allows the use of dynamic programming techniques. The agent tries to maximize the total cumulative reward. This cumulative reward is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k})$$

It is not simply the sum of instant rewards, but the discounted reward over time. Due to the  $\gamma$  it is possible to give more weight to immediate or future rewards.

## Policy

In reinforcement learning, a policy  $\pi$  defines the agent's behavior by specifying which action should be taken in each possible state [9]. Formally, a policy can be represented either as a function or as a probability distribution over actions. Two main types of policies can be distinguished. A deterministic policy  $\pi(s)$  assigns a single, fixed action to each state. In contrast, a stochastic policy  $\pi(a \mid s)$  associates a probability with each available action. The agents progressively learn an optimal policy through continuous interaction with the environment. This learning process requires collecting experience by visiting new states and testing different actions. Through this interaction, the agent refines its strategy in order to maximize the cumulative reward obtained over time. A central challenge in this process is the exploration-exploitation trade-off. Exploration encourages the agent to try new and unfamiliar actions in order to discover potentially better rewards and improve its knowledge of the environment. On the other hand, exploitation focuses on selecting actions that are already known to yield high returns, thus capitalizing on past experience. Achieving an effective balance between these two objectives is crucial for learning a robust and optimal policy.

## Value Function

Value functions are used to evaluate policies and play a fundamental role in many reinforcement learning algorithms. The state-value function  $V_{\pi}(s)$  estimates the expected cumulative reward obtained when starting from state  $s$  and following policy  $\pi$ .

$$V_{\pi}(s) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

The action-value function  $Q_\pi(s, a)$  measures the expected return after taking action  $a$  in state  $s$  and subsequently following policy  $\pi$ , are also called Q-values [22].

$$Q_\pi(s, a) = \mathbb{E}[R(s, a, s') + \gamma V_\pi(s')]$$

The central objective of reinforcement learning is to identify an optimal policy  $\pi^*$  that maximizes these value functions across all states.

## Potential and areas of application

MDPs provide the theoretical foundation for a wide range of reinforcement learning algorithms, including value iteration, policy iteration, temporal-difference learning and Q-learning. These methods exploit the mathematical structure of MDPs to iteratively improve policies, either when the environment model is known or when it must be learned from experience. As a result, MDPs play a crucial role in enabling autonomous agents to learn effective behaviors through interaction with complex and uncertain environments. Due to their strong theoretical foundation and flexibility, MDPs are extremely important across numerous application domains. In robotics and automation, for example, robots rely on MDPs to navigate uncertain and dynamic environments. Beyond robotics, MDPs are also widely applied in healthcare for treatment planning and optimization, in finance for trading strategies and risk management, and in game AI for developing intelligent and adaptive agents. This broad applicability highlights the practical significance of MDPs in real-world decision-making problems [9].

## The path to success

The primary objective of reinforcement learning is to identify an optimal policy that maximizes long-term rewards. This objective leads to one of the most fundamental results in RL theory: the Bellman Optimality Equation [26]. This equation provides a recursive definition of optimal value functions, both for states and for state-action pairs [27].

- **Optimal State Value Function  $V^*(s)$ :**

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')] \quad (2.1)$$

- **Optimal Action Value Function  $Q^*(s, a)$ :**

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) \left[ R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

The Bellman optimality equation removes any dependence on a particular policy. Instead, it establishes a self-consistency condition that optimal value functions must satisfy. Specifically, the optimal state-value function is defined as the maximum expected return achievable by selecting the best possible action in a given state. Similarly, the optimal action-value function is expressed as the immediate reward plus the highest expected future return obtainable from the subsequent state [27]. The Bellman Optimality Equation serves as the theoretical cornerstone for a wide range of optimal control algorithms used in reinforcement learning.

## 2.4.2 Types of Reinforcement Learning

Several methods have been developed to solve Markov Decision Processes (MDPs), differing mainly in how they use information about the environment. These approaches can be broadly classified into model-based and model-free methods. Model-based methods rely on a known or learned model of the environment to predict future states and rewards, allowing agents to plan their actions in advance. In contrast, model-free methods do not assume any prior knowledge of the environment and instead learn optimal behaviors directly through experience and interaction. Understanding this distinction is fundamental for analyzing how different reinforcement learning algorithms operate and adapt to uncertainty.

### Model-Based Reinforcement Learning

**Dynamic Programming Methods** Dynamic programming is a model-based approach and solves MDPs by iteratively improving value functions through:

- **Policy Iteration** alternates between policy evaluation and policy improvement until the policy converges to the optimal policy.
  - *Policy evaluation* consists of estimating the value function associated with a given policy  $\pi$ , typically by computing the expected return obtained when the agent follows that policy from each state. This step answers the question of how good the current policy is, by assessing its long-term performance in the environment.
  - *Policy improvement*, on the other hand, aims to enhance the current policy by exploiting the information obtained during policy evaluation. In this phase, the policy is updated by selecting actions that yield higher expected returns according to the evaluated

value function, resulting in a new policy that is guaranteed to be at least as good as the previous one [27].

- **Value Iteration:** updates the value function until it converges to the optimal value function.

$$V_{k+1}(s) = \max_{a \in A} \sum_{s' \in S} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

where:

- $k$  is the iteration index of the algorithm;
- $V_k(s)$  is the estimated value of state  $s$  at iteration  $k$ ;
- $V_{k+1}(s)$  is the updated value estimate at the next iteration;
- $A$  is the set of available actions;
- $S$  is the set of possible next states;
- $P(s' | s, a)$  is the transition probability to state  $s'$  given state  $s$  and action  $a$ ;
- $R(s, a, s')$  is the reward received after transitioning to  $s'$ ;
- $\gamma \in [0, 1)$  is the discount factor.

Dynamic programming breaks down complex tasks into smaller subtasks. It then models problems as workflows of sequential decisions made in discrete time steps. Each decision is made based on the possible resulting next state, since dynamic programming considers potential future states and reward signals when selecting actions. Indeed, dynamic programming is a value-based method, meaning it selects actions based on their estimated values according to a policy that aims to maximize its value function. An agent's reward  $R$  for a given action is defined as a function of that action  $a$ , the current environmental state  $s$ , and the potential next state  $s'$ :

$$R(s, a) = \sum_{s' \in S} R(s, a, s') P(s' | s, a)$$

where  $P_t(s' | s, a)$ , called the *transition probability function*, denotes the probability that the system is in state  $s'$  at time  $t+1$  when the agent chooses action  $a$  in state  $s$  at time  $t$  [9]. Dynamic programming seeks to maximize cumulative rewards by consistently selecting the best possible actions at each state, using recursive relationships such as the Bellman equations [26]. Dynamic programming theory guarantees the existence of at least one optimal stationary policy. However, classical dynamic programming techniques require full knowledge of the reward function and transition probabilities, which are generally unavailable in real-world scenarios.

## Model-Free Reinforcement Learning

**Monte Carlo methods** The Monte Carlo method assumes a black-box environment, making it a model-free approach. Since no prior knowledge of the environment is available, Monte Carlo methods rely entirely on direct interaction to gather information, so these methods are purely experience-driven. These methods sample sequences of states, actions and rewards exclusively through interaction with the environment. As a result, learning occurs through trial and error [27] rather than through predefined probability distributions or transition models. This experiential learning process enables Monte Carlo methods to adapt to unknown environments, but it also requires a large number of episodes to achieve reliable performance. Monte Carlo methods estimate value functions by computing the average return for each state-action pair based on observed episodes. Consequently, Monte Carlo methods must wait until an entire episode (or planning horizon) is completed before updating their value estimates and policies. Within this framework, two main variants can be distinguished: First-Visit and Every-Visit Monte Carlo.

- **First-Visit Monte Carlo** estimates the value of a state by averaging the returns obtained only on the first visit to that state in each episode. This avoids updating the same state multiple times within the same episode, reducing the risk of bias in the results.
- **Every-Visit Monte Carlo**, on the other hand, considers all visits to a state within each episode. The state's value is estimated by averaging the returns obtained in each visit. This method uses more data than First-Visit but may introduce greater correlation between observations.

In both variants, the value function is updated according to:

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

where  $V(s)$  is the estimated value of state  $s$ ,  $\alpha$  is the learning rate, and  $G_t$  is the actual return obtained from time step  $t$ .

**Temporal Difference learning** Temporal-Difference (TD) learning is a hybrid approach that combines the concepts of dynamic programming and Monte Carlo methods. It relies on experience to learn from interaction with the environment as Monte Carlo method, indeed is model-free approach. It also incorporates the bootstrapping idea from dynamic programming by updating value estimates incrementally, based on existing predictions and after

each transition, without waiting for the final outcome of an episode. In this way, TD learning algorithms adjust the estimated value of a state and consequently the agent's policy, incrementally after each step, enabling faster and more continuous learning compared to waiting for complete episodes. As its name suggests, a TD learning agent adjusts its policy based on the difference between the predicted reward and the reward received at each state. In other words, while both dynamic programming and Monte Carlo methods focus on the rewards obtained, TD learning additionally evaluates the prediction error between expected and observed rewards. This difference, known as the temporal difference error, is then used to update value estimates for subsequent states immediately, without waiting for the episode to terminate.

$$V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

where:

- $V(s_t)$  is the estimated value of state  $s$  at time step  $t$ ;
- $\alpha \in (0, 1]$  is the learning rate, controlling the update step size;
- $R_{t+1}$  is the reward received after transitioning from  $s_t$ ;
- $\gamma \in [0, 1)$  is the discount factor;
- $V(s_{t+1})$  is the estimated value of the next state  $s_{t+1}$ .

The temporal difference error  $\delta_t$  is defined as:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

so that the update rule can be written compactly as:

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t$$

TD learning exists in multiple variants, among which State-Action-Reward-State-Action (SARSA) and Q-learning are two of the most prominent. SARSA is an on-policy TD algorithm, meaning it evaluates and improves the same policy that it follows during learning, therefore updates the action-value function based on the action taken by the current policy:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where:

- $Q(s_t, a_t)$  is the estimated value of taking action  $a_t$  in state  $s_t$ ;



- $\alpha \in (0, 1]$  is the learning rate;
- $R_{t+1}$  is the reward received after the transition;
- $\gamma \in [0, 1)$  is the discount factor;
- $Q(s_{t+1}, a_{t+1})$  is the estimated value of the next state-action pair, where  $a_{t+1}$  is the action chosen by the **current policy**.

Q-Learning is a model-free, off-policy reinforcement learning algorithm that aims to directly approximate the optimal action-value function, independently of the policy used to generate behavior [28]. Q-Learning separates the behavior policy, which directs exploration to gather new experiences, from the target policy, which focuses on exploiting the current value estimates to guide optimal decisions. The behavior policy may include exploratory mechanisms such as  $\varepsilon$ -greedy action selection, while the target policy is implicitly defined as the greedy policy with respect to the learned action-value function. This separation allows Q-Learning to learn the optimal policy regardless of how the agent behaves during data collection, provided that sufficient exploration occurs. For this reason, the algorithm is often described as exploration-insensitive, meaning that its convergence properties do not depend on the specific exploration strategy adopted, as long as all state-action pairs are visited sufficiently often [9]. The core update rule of Q-Learning is derived from the Bellman optimality equation and is defined as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

where:

- $\alpha$  is the learning rate,
- $\gamma$  is the discount factor,
- $R_{t+1}$  is the reward received after taking action  $a_t$  in state  $s_t$ ,
- $\max_a Q(s_{t+1}, a)$  represents the maximum estimated future return obtainable from the next state.

The algorithm updates the action-value function using the maximum estimated reward of the subsequent state, allowing it to iteratively improve its policy based on expected future returns. Since Q-Learning does not require explicit knowledge of the transition probabilities or reward function, it can learn directly from raw experience, making it particularly suitable for environments where a model is unavailable or difficult to construct [29]. Once the

optimal action-value function  $Q^*(s, a)$  has been learned, the optimal policy is obtained by selecting, in each state, the action that maximizes the estimated Q-value:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

For convergence to the optimal solution, it is necessary that every state-action pair is visited infinitely often and that the learning rate decreases appropriately over time. Under these conditions, also called stochastic approximation conditions, the action-value estimates are guaranteed to converge to their optimal values [28].

### 2.4.3 Deep Q-Networks

Deep Q-Networks (DQN), introduced by Mnih et al. in 2015 [26], revolutionized reinforcement learning by demonstrating that deep neural networks could successfully approximate Q-functions in high-dimensional state spaces. Prior to DQN, Q-learning was limited to small, discrete state spaces due to its reliance on tabular representations. By integrating convolutional neural networks (CNNs) with Q-learning [26], DQN enabled end-to-end learning directly from raw sensory inputs, such as pixel-based game screens. This innovation allowed reinforcement learning agents to achieve human-level performance on several Atari 2600 games using only visual input and game scores as feedback, without any handcrafted features. The DQN algorithm addresses two key challenges that arise when combining Q-learning with neural networks: instability due to correlations in the sequence of observations and non-stationarity of the target values used for training. In standard on-line reinforcement learning, consecutive observations are highly correlated. Training a neural network on sequential samples violates the independent and identically distributed (i.i.d.) assumption required for stable stochastic gradient descent, leading to oscillations or divergence. In addition, Q-learning relies on bootstrapped targets derived from the same network being trained. As network parameters change during learning, the target values also change, resulting in a moving target problem that destabilizes training. DQN employs two critical innovations to stabilize learning [26]. The first innovation was inspired by biological mechanism, the experience replay stores agent transitions, at each time-step, in a replay buffer. In addition, during training, samples minibatches uniformly for training. This technique breaking temporal correlations between past experience, enabling more efficient use of past experience and stabilizes learning through randomized training batches. This allows gradient updates to better approximate the true ex-

pectation of the Bellman equation. The second innovation was targeting networks. This technique maintains a separate copy of the Q-network whose parameters are updated less frequently, providing more stable target values for the loss function, this help to reduces harmful feedback loops and prevents divergence. These techniques enable successful learning in complex domains such as Atari 2600 games, where the state space consists of raw pixel inputs.

**Loss Function and Optimization** The DQN loss function is designed to minimize the difference between the current Q-value estimates and the target values defined by the Bellman optimality equation. Using a target network with parameters  $\theta^-$ , the algorithm stabilizes learning by providing a fixed reference for the updates. In practice, DQN can be seen as a method that approximates the Q-function with a deep neural network and iteratively solves the Bellman optimality equation via stochastic gradient descent.

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

where:

- $L(\theta)$  is the loss function to be minimized with respect to the network parameters  $\theta$ ;
- $(s, a, r, s') \sim U(D)$  denotes that the transitions are sampled uniformly from the replay buffer  $D$ ;
- $r$  is the immediate reward received after taking action  $a$  in state  $s$ ;
- $\gamma \in [0, 1)$  is the discount factor;
- $\theta_i^-$  are the parameters of the target network, kept fixed during updates to stabilize training, at iteration  $i$ ;
- $\max_{a'} Q(s', a'; \theta_i^-)$  is the maximum Q-value over all possible next actions  $a'$ , estimated by the target network;
- $Q(s, a; \theta_i)$  is the Q-value estimated by the online network for the current state-action pair  $(s, a)$ ;
- $r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i)$  is the TD error, measuring the discrepancy between the current estimate and the bootstrapped target based on the best possible next action [5].

One of the most significant aspects of DQN is its ability to learn directly from raw sensory inputs, requiring little to no prior domain knowledge or handcrafted feature engineering. Experimental results show that the DQN agent is capable of achieving performance levels comparable to, and in some cases exceeding, those of human players across a wide range of Atari 2600 games [26]. These findings highlight the robustness of deep reinforcement learning methods, showing that complex behaviors can emerge solely from interaction with the environment and scalar reward feedback. The model’s success across diverse gaming environments indicates its potential applicability to broader decision-making problems beyond the Atari benchmark.

## 2.5 Atari-like Environments as Testbeds

Over the past decade, two-dimensional video games, especially those from the Atari 2600 game suite, have become standard benchmarks for the evaluation of reinforcement learning and artificial intelligence systems [30]. By offering high-dimensional visual inputs, diverse dynamics, and well-defined evaluation protocols within a controlled setting, these environments bridge the gap between simple toy problems and real-world applications. This section examines the characteristics and inherent challenges that make Atari-like environments particularly suitable as experimental testbeds for AI research.

### 2.5.1 Characteristics and Challenges of two-dimensional Game Environments

Atari games present diverse challenges that require different capabilities [31]. In this study, we concentrate our analysis on the Atari game Breakout as a representative benchmark environment. This choice allows us to highlight several fundamental properties shared by many Atari games suit. More in details, these environments often require strong temporal reasoning. Temporal reasoning is essential in many games, where success depends on understanding sequences of events, anticipating future states, and planning multi-step strategies. In some cases, precise timing of actions is crucial, while in others, rewards are delayed and the consequences of decisions become apparent only after multiple steps. Breakout requires predicting where the ball will land based on its current trajectory and velocity. Furthermore, the agent must plan the paddle’s movements several steps in advance, especially when the ball bounces off bricks or edges. This corresponds to the need for multi-step planning and anticipation of future states. Exploration and credit assignment pose additional challenges. Many Atari games sparse

reward structures, in which positive feedback is infrequent, making difficult for learning algorithms to discover effective strategies. Furthermore, long action sequences with delayed outcomes complicate the attribution of credit to individual actions that ultimately lead to success or failure. Partial observability is another important characteristic of several games, where the full state is not visible in a single frame. This requires agents to integrate information over time or maintain memory of past observations. In Breakout, even though the agent receives numerical information such as the position and speed of the ball, it does not know the complete future trajectory or any changes in speed due to multiple bounces. This necessitates mechanisms for temporal integration beyond simple reactive policies. Generalization across games remains a key challenge [31] for the development of general-purpose learning systems. Although deep reinforcement learning has achieved super-human performance on individual games, transferring of learned knowledge across different games remains difficult. Each game may require different visual features, temporal abstractions, and strategic reasoning, thereby testing the generality and robustness of learning algorithms. Even though Breakout doesn't directly test your ability to transfer strategies to other games, while the goal is to develop a generic algorithm, generalization remains a theoretical challenge of the Atari suite. The diversity and complexity of Atari games, combined with their standardized interface and computational tractability, make them ideal testbeds [30] for developing and evaluating artificial intelligence systems. They provide a middle ground between simple toy problems and real-world applications, offering sufficient complexity to be challenging while remaining amenable to systematic experimentation. The characteristics are particularly relevant to the framework proposed in this thesis, which aims to extract interpretable symbolic knowledge from visual observations and use this knowledge to guide learning. The structured nature of 2D games, characterized by discrete objects obeying consistent rules, provides a suitable domain for testing the hypothesis that combining perceptual processing, symbolic reasoning, and reinforcement learning can lead to more efficient, interpretable, and generalizable AI systems.

## 3 State of the Art and Challenges

Although artificial intelligence has achieved significant successes in recent years, particularly through deep learning methodologies [5], fundamental challenges persist in the development of systems that are truly intelligent, interpretable, reproducible, and trustworthy. This chapter examines the current state of the art in AI, focusing on the limitations of purely neural approaches and the emerging paradigm of neuro-symbolic integration [32]. We explore the cognitive foundations that inspire alternative approaches, and the position of the framework proposed in this thesis within the broader landscape of contemporary AI research.

### 3.1 Challenges in Symbolic Interpretation

#### 3.1.1 The Illusion of Understanding

Modern deep learning systems have achieved impressive performance on benchmark tasks, often matching or exceeding human capabilities in specific domains. Large language models such as GPT-4 and BERT demonstrate near-human accuracy in translation, question answering, and text generation, while deep neural networks excel in image classification and game-playing environments. These achievements often create the illusion of genuine understanding [7], masking fundamental limitations in how such systems process information. The impressive performance of modern AI systems has fuelled widespread optimism, with some interpretations suggesting that these models exhibit forms of intelligence or comprehension comparable to human cognition. However, this view has been critically challenged by several scholars. Bender et al. famously described large language models as “*stochastic parrots*”, emphasizing that their outputs are generated through sophisticated statistical pattern matching rather than through semantic un-

derstanding or intentional reasoning [33]. According to this critique, LLMs predict the most probable sequences of words based on training data distributions, without possessing an internal model of meaning or the world. A concrete manifestation of this limitation is the phenomenon of hallucinations, wherein language models confidently generate incorrect or nonsensical statements that are nonetheless statistically plausible [29]. While extensive research efforts aim to mitigate hallucinations through architectural improvements, or better datasets, some researchers argue that hallucinations are an intrinsic consequence of the probabilistic nature of these models [34]. This reinforces the distinction between surface-level linguistic competence and genuine understanding. Similar shortcomings emerge in applied contexts such as AI-assisted content moderation. Social media platforms including Facebook, Instagram, and X deploy deep learning systems to detect hate speech, misinformation, and policy violations at scale. Despite their efficiency, these systems frequently struggle with contextual nuance, irony, and cultural variation, resulting in both over-censorship and under-enforcement. These failures highlight the inability of current AI systems to reason symbolically about intent, meaning, and social context. Melanie Mitchell has extensively analysed these limitations, particularly in her work *A Guide for Thinking Humans* [35]. She demonstrates how contemporary AI systems, spanning conversational agents, image generation models, and image captioning architectures, systematically fail when exposed to adversarial inputs specifically designed to be trivial for humans yet confusing for machines. These failures expose the lack of commonsense reasoning and conceptual grounding in both text-based and vision-based architectures. A paradigmatic example is the Winograd Schema Challenge [36], a benchmark designed to test an AI system’s ability to resolve linguistic ambiguity using commonsense knowledge. Consider the sentence: “*The trophy did not fit in the suitcase because it was too big.*” Humans effortlessly infer that “*it*” refers to the trophy, based on an intuitive understanding of physical size and spatial relationships. In contrast, AI models often perform poorly on such tasks, achieving accuracy rates below 70% , whereas humans typically score near 100%. This gap illustrates that neural models do not reason about physical properties or causality but rather rely on shallow textual correlations. Mitchell also examines DeepMind’s game-playing agents, noting that despite their superhuman performance in specific games, these models remain extremely narrow. Each agent must be trained extensively on a single game, and the knowledge acquired does not transfer to even slightly modified environments. Some studies on the robustness of Deep Q-Networks (DQN) [37] show that minor visual perturbations, such as changing background colours or object positions, can cause complete failure. This stands in contrast to human performance, who grasp the abstract rules

of a game and adapt to superficial changes, instead, these agents memorize pixel-level patterns and reward structures without forming a conceptual understanding of the environment.

### **The Lack of World Models**

The limitations observed in language processing extend to other domains, including reinforcement learning and game-playing agents lead to a broader critique: modern AI systems lack an internal world model, like a structured representation of objects, relationships, causality, and dynamics necessary for reasoning and generalization [38]. Without such models, AI systems struggle to apply learned knowledge to novel contexts or perform commonsense reasoning reliably. Gary Marcus echoes this concern, arguing that progress in AI requires moving beyond narrow task optimization toward architectures that integrate structured knowledge and symbolic reasoning [39]. He advocates for hybrid and neuro-symbolic approaches as a promising path forward. As machine learning-based decision support systems become increasingly influential, it is essential for users to be able to critically assess the assistance they provide. A key requirement in this context is interpretability, which refers to the ability to understand and reason about a model’s outputs [40]. Despite years of research efforts, progress in this area has been relatively limited. Highly successful models such as deep neural networks, while achieving near-human accuracy in various prediction and classification tasks, largely operate as black boxes [26]. They offer little transparency regarding how and why certain features are favoured over others during training, how patterns and correlations in the training data are internally represented, and how specific computational pathways transform raw inputs into final predictions. This limitation is particularly problematic as machine learning systems increasingly influence high-stakes decisions. In such contexts, interpretability becomes essential: users must be able to understand, question, and trust the outputs of AI systems. Despite extensive research, most high-performing deep neural networks remain opaque black boxes. The tension between predictive power and understanding becomes especially apparent when models are evaluated outside their training distribution. Image classifiers may rely on irrelevant background cues rather than object-defining features; language models may generate fluent but logically inconsistent text; and agents may exploit unintended shortcuts rather than learning the intended task structure. This behavior, often referred to as shortcut learning [41], reflects the tendency of neural networks to optimize for superficial correlations that satisfy training objectives without capturing underlying conceptual structure. Whereas these representations are statistically effective, they lack the com-



positionality, systematic generalization, and explanatory clarity that characterize human cognition. As a result, modern AI systems can appear highly intelligent while remaining fundamentally brittle, uninterpretable, and unable to truly understand the problems they solve.

### 3.1.2 The Black Box Problem

The illusion of understanding is closely tied to the black-box nature of deep neural networks. The opacity of deep neural networks poses significant challenges to trust, accountability, and scientific understanding [42, 43]. Modern deep neural network architectures are highly complex nonlinear statistical models with an exceptionally large number of parameters, making direct interpretation of their decision-making virtually impossible. This inherent complexity results in a lack of transparency, as these models are unable to provide explicit and interpretable explanations for the rationale underlying their decisions, which in turn has several important implications. In high-stakes applications such as medical diagnosis, criminal justice, or autonomous vehicles [1, 42], the demand for interpretability stems from the inherent incompleteness of problem formalization. In these contexts, producing an accurate prediction alone is insufficient, as it only partially addresses the original decision-making problem. The inability to explain how and why [40] a system arrives at a given outcome raises significant ethical and legal concerns, undermining trust and accountability. Consequently, stakeholders require not only reliable predictions, but also transparent explanations that clarify the reasoning processes behind those predictions, enabling informed evaluation, justification, and responsible use of automated systems. Despite significant advances in data modelling and learning algorithms, a clear gap remains between the construction of models and the extraction of meaningful knowledge from them. Machine learning models can be represented in many ways depending on the techniques used; however, true knowledge can only be claimed when these representations are understandable to humans. Knowledge extraction inherently involves human cognitive processes, which extend far beyond purely technical considerations. Without interpretability, models risk becoming ineffective, as their output cannot be properly assessed or trusted. For this reason, interpretability should be regarded as a fundamental requirement [44] for machine learning approaches intended for real-world applications. Moreover, the opacity of neural networks significantly complicates their debugging and improvement. When a model fails, pinpointing the underlying causes and identifying effective corrective measures is often extremely challenging [45]. The absence of explicit and interpretable representations makes it difficult to determine whether errors

arise from insufficient or biased training data [40], architectural limitations, or fundamental shortcomings in the learning process. Bias in both data and models is a particularly critical concern, as it can perpetuate harmful stereotypes and produce unfair outcomes [46]. Finally, the lack of transparency also limits the scientific insight that can be gained from successful AI systems. Although building effective models is undeniably valuable from an engineering perspective, understanding the principles that govern their behavior is essential for advancing the field of artificial intelligence. Black-box models may achieve high performance on specific tasks yet fail to provide generalizable insights that could guide the development of more robust, reliable, and capable systems. In response to these concerns, a growing body of research has sought to make sense of AI systems by focusing on their interpretability and explanatory power.

### **Making AI Transparent: Principles of Explainable Machine Learning**

These limitations have spurred research into explainable artificial intelligence (XAI), which aims to make the behavior of machine learning systems more transparent and interpretable [44, 47]. The approaches to interpretable machine learning can be broadly categorized based on when interpretability is achieved. Post-hoc explanation methods attempt to interpret already trained models by constructing a secondary model that provides explanations for the original system. In contrast, inherently interpretable models are designed to be transparent from the outset [45], achieving interpretability by embedding explanatory mechanisms directly into their structures, such as decision trees and rule-based models. Additionally, hybrid approaches combine symbolic reasoning with neural learning to balance predictive performance and interpretability. Despite impressive empirical achievements, contemporary AI systems remain fundamentally limited in their capacity for genuine understanding [7], reasoning, and symbolic interpretation. Their reliance on statistical correlations, lack of internal world models, and opaque decision-making processes constrain their ability to generalize, explain, and reason about complex environments. Future research should therefore prioritize interpretability, commonsense reasoning, and the development of structured internal representations, shifting the focus from narrow benchmark performance toward robust, transparent, and truly intelligent systems.

### 3.1.3 Learning Paradigms and Explainable AI

The growing demand for explainability in artificial intelligence is not merely a response to ethical or regulatory concerns, but reflects a deeper limitation rooted in contemporary learning paradigms. Most state-of-the-art AI systems rely on statistical learning from large-scale datasets, optimizing performance objectives without explicitly representing meaning, causality, or structured knowledge. As a result, the challenge of explainability cannot be separated from the way models learn and internally represent information. This section examines Explainable Artificial Intelligence (XAI) as a response to the black-box nature of modern machine learning, highlighting both its achievements and its fundamental limitations. In doing so, it argues that explainability is inseparable from learning mechanisms and representational choices, and that post-hoc explanations alone are insufficient to address the deeper issues of understanding and reasoning in AI systems.

#### Learning Without Understanding

Modern machine learning systems, particularly deep neural networks, learn by identifying statistical regularities in data rather than by constructing explicit models of the world. Learning is typically framed as the optimization of a loss function over large datasets, with success measured primarily in terms of predictive accuracy. While this paradigm has proven highly effective in narrow domains, it does not guarantee that the learned representations correspond to meaningful concepts or causal structures [2, 35]. Consequently, models may rely on spurious correlations, dataset biases, or superficial cues that enable high performance during training but fail to generalize under distributional shifts, attributable to phenomena of this kind. This highlights the gap between statistical success and genuine conceptual understanding [41]. Moreover, because internal representations are distributed and opaque, it is difficult to determine what a model has truly learned, even when its predictions appear correct. This form of learning without understanding directly contributes to the interpretability problem. If a system lacks structured internal representations, explanations can only describe correlations in behavior rather than reasons grounded in knowledge or world models.

#### Explainable AI: Goals and Approaches

Explainable Artificial Intelligence (XAI) emerged in response to the opacity of high-performing machine learning models, particularly deep neural networks [42]. The primary goal of XAI is to enable humans to understand, trust, and effectively interact with AI systems by providing explanations of

their behavior. Interpretability is especially critical in high-stakes domains such as healthcare, finance, law, and autonomous systems, where decisions must be justified, audited, and contested when necessary [48]. Beyond practical considerations, explainability is also essential for scientific understanding, debugging, and the iterative improvement of models. However, explainability is often treated as an external requirement imposed after learning has already occurred. This framing raises a fundamental question: can meaningful explanations be produced if the learning process itself does not involve the acquisition of interpretable knowledge?

### Post-hoc Explainability

A large body of XAI research focuses on post-hoc explanation methods, which aim to interpret the behavior of trained black-box models without modifying their internal structure. Prominent examples include feature attribution techniques such as LIME (i.e., Local Interpretable Model-Agnostic Explanations) [49] and SHAP (i.e., Shapley Additive Explanations) [50], which estimate the contribution of individual input features to a model’s prediction. These approaches assist in identifying biases, facilitating debugging, and enhancing user confidence. Other approaches include saliency and attention-based methods, particularly in computer vision and NLP (i.e., natural language processing), which highlight regions of the input that most strongly influence model outputs [51]. For instance, in image classification, saliency maps can indicate the specific areas of an image that influenced a particular prediction. Surrogate models provide another strategy, approximating complex models with simpler, interpretable ones such as decision trees or linear regressions [52]. This method aids in interpreting the overall behavior of black-box models, though its accuracy in reflecting the original model may be constrained in highly nonlinear decision domains. While these methods offer valuable insights into model behavior, they remain fundamentally limited. Post-hoc explanations describe *"how"* a model behaves locally, but not *"why"* it behaves that way in terms of underlying concepts or causal reasoning. As a result, explanations may be unstable, incomplete, or misleading, especially when models operate in highly nonlinear or out-of-distribution settings [53].

### Intrinsic Interpretability

In contrast to post-hoc approaches, intrinsically interpretable models aim to achieve transparency by design. These models, including decision trees, rule-based systems, linear models, and symbolic representations, provide explicit reasoning structures that can be directly inspected and understood by hu-

mans [40]. Intrinsic interpretability is often achieved by constraining model complexity or imposing structured representations. However, this typically comes at the cost of reduced flexibility or performance when compared to deep neural networks. This trade-off has historically limited the adoption of interpretable models in complex real-world tasks. Recent research has therefore explored hybrid approaches that integrate neural learning with symbolic or structured components, seeking to balance expressivity with transparency [4]. These approaches suggest that interpretability need not be an afterthought, but can emerge naturally from models that learn structured, compositional representations.

### **Why XAI is Not Enough**

Despite significant progress, XAI methods cannot fully compensate for the absence of explicit internal world models. Explanations that operate solely at the level of input-output correlations fail to capture higher-level concepts such as object identity, causality, goals, and physical constraints. Without these structures, explanations remain shallow and task specific. This limitation has led several researchers to argue that explainability must be addressed at the level of representation and learning, rather than through post-hoc analysis alone [54]. In other words, a system can only explain what it has explicitly represented. From this perspective, the challenge of explainable AI is inseparable from broader questions about how knowledge is acquired, structured, and used for reasoning. This insight motivates a shift toward cognitively inspired approaches that emphasize structured representations, inductive biases, and internal world models topics that will be explored in the following section.

## **3.2 Making Sense of AI: Toward Explainability**

### **3.2.1 Cognitive Foundations**

The limitations of purely data-driven approaches have led researchers to reconsider insights from cognitive science and developmental psychology. Understanding how humans acquire knowledge, form concepts, and reason about the world may provide valuable principles for building more effective and interpretable AI systems.

### **3.2.2 Human World Modelling and the Origins of Structured Cognition**

Human intelligence is characterized by the ability to form internal models of the external world; structured representations that capture the objects, the relationships, and the causal mechanisms that govern observable phenomena. These models enable prediction, planning, and reasoning beyond immediate perceptual experience. The question of how these models are acquired, has been central to cognitive science and developmental psychology. The Core Knowledge theory, developed primarily through research by Elizabeth Spelke, Renée Baillargeon [55], and colleagues, proposes that human cognition is founded on a small set of innate knowledge systems that emerge early in development and provide the foundation for subsequent learning. These core systems are domain-specific, addressing fundamental aspects of the physical and social world, and are hypothesized to be evolutionarily ancient, shared with other species. The existence of core knowledge systems suggests that learning is not a completely domain-general process of association formation, but rather involves specialized mechanisms tuned to specific aspects of the environment. This perspective contrasts with blank slate views of learning and suggests that effective AI systems may benefit from incorporating analogous inductive biases.

### **3.2.3 Core Knowledge Theory: Cognitive Principles and Psychological Foundations**

Core knowledge refers to a set of foundational cognitive systems that emerge remarkably early in human development, often within the first year of life. These systems are not merely innate facts; rather, they consist of organizational principles that guide infants' attention, interpretation of experience, and learning [55]. These systems provide initial structures for interpreting experience in specific domains. In other words, they provide a structured framework that allows infants to extract meaning and patterns before formal instruction or extensive experience. Spelke and Kinzler highlight that traditional views of human cognition have often polarized between two extremes. On one hand, some scholars have described the mind as a general-purpose learning system, capable of absorbing and adapting to any kind of experience. On the other hand, evolutionary psychologists have argued for a large number of domain-specific systems, each evolved to handle a particular cognitive task. Core knowledge represents a middle ground: humans appear to possess a limited number of specialized, but flexible, cognitive systems, each tuned to certain types of information, which together form the foundation

for more complex thought and learning. One of the most extensively studied core systems is object representation. From the earliest months of life, infants show an understanding that physical objects behave in predictable ways. They expect objects to be cohesive, moving as unified wholes; continuous, persisting even when hidden from view, and solid, unable to pass through one another. Violations of these expectations, such as an object seeming to vanish into thin air, elicit surprise and heightened attention, suggesting that infants are actively engaging in reasoning about the physical world. This system also has limits: infants can typically track only a few objects at a time, usually around three or four, a constraint that persists into adulthood when attention is stretched [?]. Interestingly, these principles appear to be universal, observable even in remote cultural groups like the Pirahã of the Amazon, whose language and counting systems differ dramatically from those commonly studied in western contexts [?]. Alongside object representation, infants develop a number sense that allows them to perceive, compare, and manipulate quantities. This system is supported by two complementary mechanisms. The Approximate Number System (ANS) allows infants to estimate quantities, with precision decreasing as numbers grow larger, and the object-tracking system enables exact representation of small sets, typically up to three or four items [?]. Together, these systems form the basis for early arithmetic reasoning and provide a foundation for later formal mathematical learning. Equally important is the spatial system, which allows infants to encode information about locations, distances, and geometric relationships. This knowledge supports navigation, understanding of containment and support, and the prediction of where objects might be in space. Spatial reasoning is not just a precursor to interacting with the physical world, it underlies more sophisticated cognitive processes later in life, including problem-solving and abstract thinking. Finally, infants possess a system for agent representation, which enables them to understand goal-directed behavior and social interactions [?]. Unlike objects, agents do not need to be cohesive or move continuously to be recognized as entities. Instead, infants interpret their actions as driven by goals, often expecting that agents will act efficiently to achieve desired outcomes. They are sensitive to contingent and reciprocal interactions, as well as social cues such as gaze direction, which allow them to anticipate intentions and understand others' actions. This system forms the basis of early social cognition and plays a role in moral reasoning and cooperative behavior throughout life. Across diverse domains, core knowledge systems share several defining characteristics. They emerge early in development, operate on specialized types of entities and their interrelations, and provide a foundational framework that supports the acquisition of increasingly complex knowledge [55]. When

the principles underlying these systems are violated, they elicit surprise and heightened attention, thereby promoting enhanced learning and cognitive engagement. Importantly, core knowledge should not be understood as a set of static, preloaded facts; rather, it consists of flexible principles that guide attention, interpretation, and learning from experience. Moreover, studying these systems highlights their evolutionary roots and universality, offering insights not only into human development but also into the ways cognition is shaped by both biology and experience.

#### **3.2.4 Definition and Relevance for World Understanding**

#### **3.2.5 Cognitive Models and Symbolic Representation**

### **3.3 Positioning of the Proposed Framework**



## 4 Framework Design - Proposed Framework

### 4.1 Conceptual Foundations

#### 4.1.1 Evolution of the Idea

### 4.2 Proposed Implementation

#### 4.2.1 Segmentation

#### 4.2.2 Symbolic System as Knowledge Construction Module

#### 4.2.3 From Symbolic Rules to a Playable Environment

#### 4.2.4 Generic Environment as a Transitional Representation

#### 4.2.5 Deep Q-Network in a Domain-Agnostic Environment

# 5 Implementation

This chapter describes the concrete implementation of the proposed framework, detailing the end-to-end processing pipeline, the architectural design choices, and the implementation of each system module. The framework integrates visual perception, symbolic reasoning, and reinforcement learning into a unified and modular architecture capable of discovering structure, causality, and control strategies from raw sensory data, validated in a dynamic game environment.

## 5.1 End-to-End Pipeline

The implemented system follows a fully automated end-to-end pipeline that processes raw visual observations and transforms them into symbolic knowledge and action policies. The pipeline is designed to progressively increase the abstraction level of information, moving from pixels to semantic objects, then to symbolic events, and finally to decision-making strategies.

The pipeline operates through the following stages:

- **Visual input acquisition**

Frames are captured from the environment or loaded from logged gameplay sessions, each frame represents the full observable state of the environment at a given time step.

- **Image preprocessing**

Input images are resized to a fixed spatial resolution and normalized using standard ImageNet statistics [56]. This ensures numerical stability and compatibility with convolutional neural networks pretrained or inspired by standard vision architectures.

- **Semantic segmentation**

A deep convolutional model assigns a semantic class to each pixel, producing dense segmentation masks that identify relevant entities such as the ball, paddle, bricks, and background.

- **Patch extraction and object tracking**

Segmented regions are converted into patches and tracked across consecutive frames, forming persistent object hypotheses with temporal continuity.

- **Symbolic abstraction and reasoning**

Visual objects are mapped into symbolic entities described by properties, events, and inferred rules. A population-based heuristic process evaluates alternative symbolic explanations and selects the most coherent ones.

- **Reinforcement learning interaction**

A reinforcement learning agent interacts with the environment through a generic symbolic interface. The rewards are derived from intrinsic measures of causality, novelty, and event generation.

- **Logging and persistence**

Intermediate representations, learned models, and symbolic populations are saved, enabling reproducibility, debugging, and incremental refinement.

This pipeline enables learning and reasoning without relying on handcrafted semantic labels or domain-specific reward functions. While semantic labels are used internally by the segmentation module to generate class maps, they are not, actually, provided as supervision for symbolic reasoning or reward assignment; the agent and the symbolic engine operate purely on observed state changes and event structures.

## 5.2 General Architecture of the Framework

### 5.2.1 Design Choices

The architecture of the framework is guided by four fundamental design principles, each contributing to its flexibility, robustness, and transparency.

The first principle, modularity, ensures that each component of the system is implemented as an independent, self-contained module with clearly defined interfaces. This design allows vision, symbolic reasoning, and reinforcement learning components to be developed, tested, or replaced in isolation, facilitating experimentation and incremental improvements without disrupting the functionality of the overall system. Domain agnosticism represents a second cornerstone: the symbolic reasoning and reinforcement learning modules

are intentionally designed to operate over arbitrary simulations, without assuming any prior knowledge of game mechanics, object semantics, or reward structures. This capability allows the framework to generalize across diverse environments and tasks, enhancing its adaptability and reducing the need for manual engineering. A third guiding principle is incremental knowledge construction; whereby symbolic knowledge is not static but continuously refined. As the system observes additional trajectories, it dynamically discovers new events, rules, and abstractions, progressively building a richer internal model of the environment. This incremental learning mechanism mirrors human-like cognitive processes, allowing the agent to improve its understanding over time. Finally, interpretability sets the framework apart from conventional end-to-end black-box approaches. By exposing intermediate symbolic representations, including object properties, causal relationships between events, and inferred rules, the system allows researchers and practitioners to inspect, analyse, and validate the agent’s reasoning process. Importantly, these symbolic components do not directly influence the DQN agent’s decisions; rather, they provide a means to quantify how much the symbolic model has captured about the environment. This transparency not only facilitates debugging and knowledge extraction but also provides a foundation for explainable AI, ensuring that the agent’s decisions can be understood and trusted.

### 5.2.2 Selected Techniques and Algorithms

The framework combines several established techniques in a novel configuration:

- U-Net [21] convolutional architecture for pixel-wise semantic segmentation.
- Heuristic population-based symbolic induction for rule discovery.
- Automatic state and event extraction based on attribute introspection.
- Deep Q-Learning [26] for policy learning.
- Intrinsic motivation mechanisms, including curiosity and causal impact rewards.

The interaction between these techniques allows the agent to learn meaningful behaviors even in the absence of explicit task rewards.

### 5.2.3 Modular Structure of the System

#### Video Acquisition and Preprocessing Module

The video acquisition module is responsible for efficiently handling visual input data and preparing it for subsequent processing stages. It supports both online acquisition of frames from live simulations and the loading of previously recorded gameplay logs. Each frame is resized to a fixed resolution of  $120 \times 70$  pixels to ensure a consistent input size for the neural network and is converted into a normalized tensor to facilitate stable training. Corresponding segmentation masks are loaded as integer-valued images, where each pixel represents a semantic class identifier, allowing the network to learn to associate spatial regions with meaningful object categories. A custom dataset class encapsulates the logic for pairing each image with its corresponding mask, while PyTorch data loaders handle batching, shuffling, and splitting into training and validation sets. A fixed random seed is used to ensure reproducibility of the dataset splits, which is crucial for consistent evaluation of model performance. By abstracting low-level data handling, this module provides a clean interface for the feature extraction stage, allowing the focus to remain on the learning task rather than on preprocessing logistics.

#### Feature Extraction Module

The feature extraction module performs semantic segmentation using a U-Net architecture [21], which is highly effective for dense pixel-wise prediction tasks. The network follows a symmetric encoder-decoder design: the encoder progressively extracts hierarchical feature representations through convolutional blocks and downsampling, while the decoder reconstructs spatial resolution using transposed convolutions and skip connections. Skip connections between corresponding encoder and decoder layers enable the network to recover fine-grained spatial details lost during downsampling, allowing for precise localization of object boundaries while maintaining contextual awareness. Each convolutional block consists of two convolutional layers with  $3 \times 3$  kernels, followed by batch normalization and ReLU activation. Batch normalization stabilizes training by normalizing intermediate activations, and ReLU introduces non-linearities that allow the network to model complex patterns. The final output layer uses a  $1 \times 1$  convolution to produce one channel per semantic class, enabling per-pixel class predictions. Training is conducted with a multi-class cross-entropy loss, optimized using the Adam optimizer. To improve stability and generalization, the training process incorporates data augmentation, input normalization, small batch sizes, and early stopping based on validation loss. Evaluation metrics in-

clude pixel accuracy and mean Intersection over Union (mIoU) [20], which quantify agreement between predicted and ground-truth segmentation maps. Overall, this module abstracts the pipeline from raw video frames to class-labeled segmentation maps, producing dense semantic representations that form the foundation for downstream tasks. Its design supports reproducible training and efficient inference, combining robust data handling with a well-established deep learning architecture.

### **Symbolic Reasoning Module**

The symbolic reasoning module converts low-level perceptual outputs into high-level symbolic representations. Each segmented object is transformed into a symbolic entity characterized by:

- A unique identifier.
- A set of observable properties (e.g., position, ???)
- A history of detected events.
- Associations with inferred behavioral rules.

Object persistence across frames is handled through patch tracking, allowing the system to reason about temporal continuity and object identity.

Rather than injecting predefined domain knowledge, the system employs a heuristic initialization process that generates a population of candidate symbolic explanations. Each individual in the population represents a distinct hypothesis about how objects behave and interact over time. Individuals are evaluated based on their ability to explain observed events while minimizing unexplained changes. Scoring functions reward coherence, temporal consistency, and explanatory completeness. The best-performing individuals are retained, serialized, and summarized into higher-level prototypes. These prototypes act as emergent core knowledge, capturing regularities such as object dynamics or interaction patterns without explicit supervision.

### **Reinforcement Learning Agent**

The reinforcement learning agent is implemented using a fully generic symbolic environment designed to work across different simulations without requiring manually engineered features.

The environment, automatically, through a specific module, extracts all observable numerical and boolean attributes from the simulation object, forming a state representation. Each attribute represents an observable property,

such as positions, velocities. Events are detected by comparing consecutive states and identifying significant changes in attribute values. A causal event tracker groups events occurring within a temporal window into causal chains. Longer chains yield higher intrinsic rewards, encouraging the agent to discover action sequences that produce rich and structured dynamics. Effectively, the agent is rewarded not only for individual events but also for sequences of events that demonstrate causal influence, promoting more complex behaviors.

The agent is trained using Deep Q-Learning with experience replay, target networks, and  $\varepsilon$ -greedy exploration. All these techniques are introduced to improve stability and efficiency during training, especially:

- **Experience Replay:** Transitions (state, action, reward, next state, done) are stored in a buffer and sampled randomly in batches for training. This breaks correlations between consecutive experiences, reduces variance, and allows the agent to reuse past experiences multiple times, improving sample efficiency.
- **Target Networks:** A separate target Q-network is maintained and updated periodically with the weights of the main Q-network. This provides a stable target for Q-value updates, mitigating oscillations and divergence that can occur when the network is updated using its own rapidly changing estimates.
- **$\varepsilon$ -Greedy Exploration:** Actions are selected according to an  $\varepsilon$ -greedy policy, where with probability  $\varepsilon$  a random action is chosen to encourage exploration, and with probability  $1 - \varepsilon$  the action with the highest predicted Q-value is selected. Over time,  $\varepsilon$  decays to favor exploitation of the learned policy while still maintaining some exploration to prevent local optima.

A central aspect of the implementation is the use of intrinsic reward signals. The total reward at each time step is composed of several components:

- **Event-based reward:** Proportional to the number of detected events and the length of their causal chains.
- **Causal impact bonus:** Measures how much the outcome of an action differs from a no-op baseline, encouraging interventions that meaningfully affect the environment.
- **Curiosity bonus:** Derived from the prediction error of a learned forward dynamics model, which estimates the next state given the current state and action.

- **Event density bonus:** Optionally rewards large overall changes in the state vector, promoting exploration of diverse configurations.

Formally, the total reward  $R_{\text{total}}$  can be expressed as:

$$R_{\text{total}} = R_{\text{event}} + w_{\text{causal}} R_{\text{causal}} + w_{\text{curiosity}} R_{\text{curiosity}} + w_{\text{density}} R_{\text{density}} \quad (5.1)$$

This reward structure allows the agent to learn even in environments with sparse or absent external rewards.

#### 5.2.4 Optimizations and Efficiency Considerations

The implementation of the framework incorporates several strategies aimed at improving both computational efficiency and the stability of the training process. One of the key measures is early stopping during the segmentation model training. By monitoring the validation loss and halting training when improvements plateau, the framework prevents overfitting while reducing unnecessary computational load. In the reinforcement learning component, the dimensionality of the observation vector is intentionally limited to reduce computational overhead and accelerate training. Although the state extractor is capable of detecting and encoding numerous attributes from the simulation, only a small subset of numerical features is included in the state vector passed to the agent. This reduces the computational overhead per step and accelerates training. Another optimization is the adaptive update of the target network in the DQN agent. Early in training, the target network is updated more frequently to promote faster convergence, while updates become less frequent as the agent stabilizes. This balance between rapid learning and stable evaluation mitigates the risk of oscillations in Q-values. Additionally, the agent uses a replay buffer from which moderate-sized batches are sampled for training. This approach mitigates correlations between consecutive transitions, allows efficient reuse of past experiences, and balances gradient accuracy with computational cost. Collectively, these optimizations allow the framework to remain tractable despite its multi-layered architecture. By combining standard deep learning practices with domain-agnostic symbolic reasoning techniques, the system achieves a balance between expressiveness and computational efficiency, ensuring that training and evaluation can be conducted on realistic hardware resources.



## 6 Experimental Evaluation

### 6.1 Datasets and Testing Scenarios

#### 6.1.1 Evaluation Metrics: Accuracy, Interpretability, Scalability

### 6.2 Experimental Results

#### 6.2.1 Comparison with Existing Methods

#### 6.2.2 Critical Discussion of the Results

## **7 Applications and Future Perspectives - Conclusion**

### **7.1 Future Directions**

#### **7.1.1 Potential Applications of the Framework**

#### **7.1.2 Possible Extensions of the Work**

#### **7.1.3 Open Challenges and Future Research Directions**

### **7.2 Summary of Contributions, Impact of the Research, and Final Reflections**

# References

# Bibliography

- [1] B. Liang, Y. Wang, and C. Tong, “Ai reasoning in deep learning era: From symbolic ai to neural-symbolic ai,” *Mathematics*, vol. 13, no. 11, p. 1707, 2025.
- [2] G. Marcus, “Deep learning: A critical appraisal,” *arXiv preprint arXiv:1801.00631*, 2018. [Online]. Available: <https://arxiv.org/abs/1801.00631>
- [3] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural Networks*, vol. 61, pp. 85–117, 2015.
- [4] A. S. d’Avila Garcez, L. C. Lamb, and D. M. Gabbay, *Neural-Symbolic Cognitive Reasoning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-540-73246-4>
- [5] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015. [Online]. Available: <https://doi.org/10.1038/nature14539>
- [6] A. S. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *AAAI Spring Symposia*, Mar. 2015, pp. 18–21. [Online]. Available: <https://www.aaai.org/Library/Symposia/Spring/ss15-01.php>
- [7] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, “Building machines that learn and think like people,” *Behavioral and Brain Sciences*, vol. 40, p. e253, 2017.
- [8] L. P. Kaelbling, M. L. Littman, and A. W. Moore, “Reinforcement learning: A survey,” *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

- [9] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
- [10] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [11] H. Bay, T. Tuytelaars, and L. Van Gool, “Surf: Speeded up robust features,” in *European Conference on Computer Vision (ECCV)*. Springer, 2006, pp. 404–417.
- [12] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.
- [13] C. Harris and M. Stephens, “A combined corner and edge detector,” in *Proceedings of the Alvey Vision Conference*, 1988, pp. 147–151.
- [14] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *European Conference on Computer Vision (ECCV)*. Berlin, Heidelberg: Springer Berlin Heidelberg, May 2006, pp. 430–443.
- [15] G. Van Houdt, C. Mosquera, and G. Nápoles, “A review on the long short-term memory model,” *Artificial Intelligence Review*, vol. 53, no. 8, pp. 5929–5955, 2020. [Online]. Available: <https://doi.org/10.1007/s10462-020-09825-6>
- [16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf)
- [17] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014, submitted on 4 Sep 2014; last revised 10 Apr 2015 (v6). [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>

- [19] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015.
- [21] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention - MICCAI 2015*, ser. Lecture Notes in Computer Science, N. Navab, J. Hornegger, W. Wells, and A. Frangi, Eds., vol. 9351. Springer, Cham, 2015, pp. 234–241. [Online]. Available: [https://doi.org/10.1007/978-3-319-24574-4\\_28](https://doi.org/10.1007/978-3-319-24574-4_28)
- [22] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Englewood Cliffs: Prentice-Hall, 1995.
- [23] M. Minsky, “A framework for representing knowledge,” in *MIT-AI Laboratory Memo 306*, Jun. 1974. [Online]. Available: <http://hdl.handle.net/1721.1/6160>
- [24] T. R. Gruber, “A translation approach to portable ontology specifications,” *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993. [Online]. Available: <https://doi.org/10.1006/knac.1993.1008>
- [25] F. Baader, Ed., *The Description Logic Handbook: Theory, Implementation and Applications*. Cambridge, UK: Cambridge University Press, 2003.
- [26] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [27] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2015.
- [28] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. [Online]. Available: <https://doi.org/10.1007/BF00992698>
- [29] L. Huang *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM*

- Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, Jan. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3703155>
- [30] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The arcade learning environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013. [Online]. Available: <https://doi.org/10.1613/jair.3912>
  - [31] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling, “Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents,” *Journal of Artificial Intelligence Research*, vol. 61, pp. 523–562, 2018. [Online]. Available: <https://doi.org/10.1613/jair.5699>
  - [32] A. S. d’Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Spranger, and S. N. Tran, “Neuro-symbolic artificial intelligence: The state of the art,” *arXiv preprint arXiv:1905.06088*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.06088>
  - [33] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’21)*. Virtual Event, Canada: Association for Computing Machinery, 2021, pp. 610–623. [Online]. Available: <https://doi.org/10.1145/3442188.3445922>
  - [34] S. Banerjee, A. Agarwal, and S. Singla, “Llms will always hallucinate, and we need to live with this,” *arXiv preprint arXiv:2409.05746*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.05746>
  - [35] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT ’21)*. Virtual Event, Canada: Association for Computing Machinery, 2021, pp. 610–623. [Online]. Available: <https://doi.org/10.1145/3442188.3445922>
  - [36] H. J. Levesque, E. Davis, and L. Morgenstern, “The winograd schema challenge,” in *Proceedings of the 13th International Conference on Principles of Knowledge Representation and Reasoning (KR 2012)*. AAAI Press, 2012, pp. 552–561. [Online]. Available: <https://www.aaai.org/ocs/index.php/KR/KR12/paper/view/4492>

- [37] X. Qu *et al.*, “Minimalistic attacks: How little it takes to fool a deep reinforcement learning policy,” *arXiv preprint arXiv:1911.03849*, 2020. [Online]. Available: <https://arxiv.org/abs/1911.03849>
- [38] M. Mitchell, “Llms and world models - part 1,” <https://aiguide.substack.com/p/llms-and-world-models-part-1>, 2023, accessed: 2026-02-28.
- [39] G. Marcus, “The next decade in ai: Four steps towards robust artificial intelligence,” *arXiv preprint arXiv:2002.06177*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.06177>
- [40] C. Molnar, *Interpretable Machine Learning*. Lulu Press, 2020. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- [41] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020. [Online]. Available: <https://doi.org/10.1038/s42256-020-00257-z>
- [42] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” *arXiv preprint arXiv:1702.08608*, 2017. [Online]. Available: <https://arxiv.org/abs/1702.08608>
- [43] C. Molnar, M. T. Ribeiro, and W. Samek, “Interpreting black-box models: A review on explainable artificial intelligence,” *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–37, 2020. [Online]. Available: <https://link.springer.com/article/10.1007/s12559-023-10179-8>
- [44] M. Du, N. Liu, and X. Hu, “Techniques for interpretable machine learning,” *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–37, 2020. [Online]. Available: <https://doi.org/10.1145/3359786>
- [45] A. Vellido, J. D. Martín-Guerrero, and P. J. G. Lisboa, “Making machine learning models interpretable,” in *Proceedings of the 2012 International Conference on Artificial Neural Networks (ICANN)*. Springer, 2012, pp. 23–29. [Online]. Available: [https://doi.org/10.1007/978-3-642-31537-4\\_4](https://doi.org/10.1007/978-3-642-31537-4_4)
- [46] P. Radanliev, “Intelligenza artificiale: Riflessioni sul passato e sguardo al prossimo cambio di paradigma,” *Journal of Experimental & Theoretical Artificial Intelligence*, vol. 37, no. 7, pp. 1045–1062, 2025. [Online]. Available: <https://doi.org/10.1080/0952813X.2024.2323042>



- [47] S. Chakraborty *et al.*, “Interpretability of deep learning models: A survey of results,” in *2017 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computed, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCCom/IOP/SCI)*, San Francisco, CA, USA, 2017, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/UIC-ATC.2017.8397411>
- [48] Z. C. Lipton, “The mythos of model interpretability,” *Communications of the ACM*, vol. 61, no. 10, pp. 36–43, 2018. [Online]. Available: <https://doi.org/10.1145/3233231>
- [49] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144. [Online]. Available: <https://doi.org/10.1145/2939672.2939778>
- [50] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions>
- [51] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep inside convolutional networks: Visualising image classification models and saliency maps,” *arXiv preprint arXiv:1312.6034*, 2014. [Online]. Available: <https://arxiv.org/abs/1312.6034>
- [52] M. Craven and J. Shavlik, “Extracting tree-structured representations of trained networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 1996, pp. 24–30.
- [53] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, “Sanity checks for saliency maps,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 9505–9515. [Online]. Available: <https://arxiv.org/abs/1810.03292>
- [54] Y. Bengio, Y. LeCun, and G. Hinton, “Deep learning for ai,” *Communications of the ACM*, vol. 64, no. 7, pp. 58–65, Jul. 2021. [Online]. Available: <https://doi.org/10.1145/3448250>
- [55] E. S. Spelke and K. D. Kinzler, “Core knowledge: Cognitive principles and psychological foundations,” *Developmental Science*,

- vol. 10, no. 1, pp. 89–96, 2007. [Online]. Available: <https://doi.org/10.1111/j.1467-7687.2007.00569.x>
- [56] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2009, pp. 248–255. [Online]. Available: <https://doi.org/10.1109/CVPR.2009.5206848>

# **A Additional Technical Details**

## **B    Code Snippets or Pseudocode of Relevant Modules**

## **C   Dataset Specifications or Experimental Settings**